# 第0章: はじめに — スマホアプリを作って公開しよう！

> **「Webサイトは作れた。次はApp StoreやGoogle Playに並ぶ"本物のスマホアプリ"を作ってみたい。」**
>
> このチュートリアルでは、**React Native（リアクト・ネイティブ）** という技術を使って、iPhone でも Android でも動く**書籍管理アプリ**をゼロから作り、最終的に**アプリストアで世界に公開する**ところまでを体験します。完全な初心者の方を想定して、一つひとつの言葉・コード・コマンドをすべて丁寧に解説していきます。
>
> なお、文章の中の `バッククォート（\`）` で囲まれた部分` は「コード（プログラムの一部）」を表す印です。プログラミング用語や、コマンド・ファイル名などをこの記号で囲って強調しています。

---

## 0. はじめる前に — 「そもそも何？」を全部解説

この章を読み始める前に、まず**スマホアプリ開発の最も基礎的な用語**を解説します。「もう知ってるよ」という方は読み飛ばして「1. この教材の目的と対象読者」から始めてください。逆に「これからプログラミングを始めます」という方は、ここを丁寧に読むと後の章が驚くほどスムーズに進みます。

> **記号の読み方の予備知識:** プログラミングでは見慣れない記号がたくさん出てきます。よく出るものを先にメモしておきましょう。- `"..."` ダブルクォート（double quote、二重引用符）。文字列を囲む記号。- `'...'` シングルクォート（single quote、一重引用符）。同じく文字列を囲む記号。- `` `...` `` バッククォート（back quote）／テンプレートリテラル。文字列を囲みつつ `${変数}` の埋め込みもできる。- `;` セミコロン（semicolon）。「文（命令）の終わり」を表す印。- `,` カンマ（comma）。値や項目の区切り。- `{ }` 中カッコ（波カッコ、curly braces）。コードの「ブロック」やオブジェクトを囲む。- `[ ]` 角カッコ（square brackets）。配列（リスト）を囲む。- `( )` 丸カッコ（parentheses）。関数の引数や、計算の優先順位に使う。- `<...>` 山カッコ（angle brackets）。React/React Native では「コンポーネント（画面部品）」を書くときに使う。例: `<Text>こんにちは</Text>` - `//` 行コメント。`//` から行末までは説明文として扱われ、プログラムとしては実行されない。- `/* ... */` ブロックコメント。複数行をまとめてコメントにする。

### 0.1 プログラミングとは何か

**プログラミング（programming）** とは、コンピュータ（ここではスマートフォン）に「やってほしいこと」を**手順書（プログラム）** として書く作業です。コンピュータは指示された通りにしか動かないので、何をしてほしいかを「コンピュータが理解できる言葉（プログラミング言語）」で書く必要があります。

身近な例で言うと、料理のレシピや、家具の組立説明書に近いものです。「(1)小麦粉を200g計る → (2)ボウルに入れる → (3)水を加える」のように、**順番と条件**を厳密に書くことで、誰がやっても（コンピュータが何度実行しても）同じ結果になるようにします。

> **覚えておきたい3要素:** - **プログラム（program）**：コンピュータへの指示書そのもの。「ソースコード（source code、元になるコードという意味）」「コード」とも言う。 - **プログラミング言語（programming language）**：その指示書を書くための言語。本書では **TypeScript（タイプスクリプト）** を使う。 - **プログラマー（programmer）/エンジニア（engineer）**：プログラムを書く人。

## 0.2 「アプリ」とは何か — Webサイトとの違い

スマホには2種類の"見るもの"があります。

| 種類 | 何か | 例 |
| --- | --- | --- |
| **Webサイト/Webアプリ** | ブラウザ（Safari, Chrome）で開いて見るページ | Googleで検索した結果のページ、ニュースサイト |
| **アプリ（ネイティブアプリ）** | App Store / Google Play からインストールして、ホーム画面のアイコンから起動するもの | LINE、Instagram、電卓、カメラ |

**アプリ（application、アプリケーションの略）** とは、スマホに「インストール（取り込んで使える状態にすること）」して使うソフトウェアです。ブラウザを経由せず、ホーム画面のアイコンを押すと直接起動します。本書のゴールは、この「アプリ」を自分で作り、ストアに公開することです。

> **「ネイティブ（native）」とは？**「その端末に生まれつき備わった」という意味の英単語です。iPhone なら iOS、Android ならその OS が直接実行できる形式のアプリを「ネイティブアプリ」と呼びます。ブラウザの上で動くWebアプリより、カメラ・GPS・通知などスマホの機能を細かく使えるのが特徴です。

## 0.3 ソースコード・ファイル・拡張子の超基礎

プログラムは **テキストファイル**（text file：文字だけが入っているファイル）に書きます。Word文書のような装飾（太字・色など）を持たない、純粋な文字だけのファイルです。ファイルの末尾には **拡張子（かくちょうし、extension）** という「.（ドット）＋数文字」が付きます。拡張子を見れば「中身がどんな種類のファイルか」が分かるようになっています。

| 拡張子 | 何のファイルか | このチュートリアルでの登場場面 |
| --- | --- | --- |
| `.js` | JavaScript（プログラミング言語）を書くファイル | 設定ファイルなどで時々登場 |
| `.ts` | TypeScript（JavaScriptに型を足した言語）を書くファイル | ロジック（処理）を書くときに使う |
| `.tsx` | TypeScript＋JSX（画面を組み立てる記法）を書くファイル | 画面（コンポーネント）を作るとき頻出 |
| `.json` | 設定情報やデータをやり取りする形式のファイル | `package.json`、`app.json` などで頻出 |
| `.md` | Markdown（このチュートリアル自身の形式）。説明文用 | 教材ファイルの拡張子 |

▼ **例:** `index.tsx` という名前のファイルは、「中身が TypeScript＋JSX で書かれた、`index` という名前のファイル」を意味します。ファイル名（`index`）と拡張子（`.tsx`）は「.」（ドット）で区切られています。

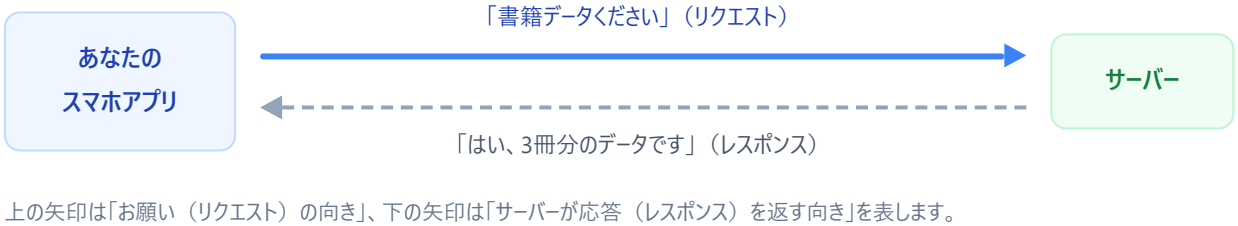
> **`.tsx` の「x」って何？** JSX（ジェイエスエックス、JavaScript XML）という「画面の見た目を `<Text>...</Text>` のようなタグで書く記法」を含むファイルに付ける印です。React / React Native で画面を作るときに使います。

## 0.4 スマホアプリはどう動いているのか

あなたがアプリのアイコンをタップすると、スマホの中で次のことが起きています。

1. **OS（オーエス、Operating System）** がアプリを起動する。OS とは iPhone の「iOS」、Android の「Android」のような"スマホ全体を動かす土台ソフト"のこと。
2. アプリが**画面を描画（びょうが＝絵を描くこと、rendering）** する。ボタンや文字を画面に並べる。
3. ユーザーが**タップ（指で触れる操作）** する。
4. アプリがその操作に反応して、画面を変えたり、データを保存したり、インターネット通信をしたりする。

多くのアプリは、自分のスマホの中だけで完結せず、**インターネットの向こうにあるサーバー（server）** とデータをやり取りします。




上の矢印は「お願い（リクエスト）の向き」、下の矢印は「サーバーが応答（レスポンス）を返す向き」を表します。

**サーバー（server、サーブ＝提供する者）** とは、24時間ずっと電源が入っていて「リクエストが来たら何かを返す」役割のコンピュータです。本書では、このサーバー側を **Supabase（スーパベース）** というサービスに任せます（第5章で解説）。

## 0.5 フロントエンドとバックエンドの違い

アプリ開発でもよく「**フロントエンド／バックエンド**」という言葉を使います。「フロント（front）」は前、「バック（back）」は後ろ、「エンド（end）」は端という意味で、ユーザーから見える「前側」とサーバーで動いている「後ろ側」を表しています。

| 用語 | 意味 | 担当する技術（本書で使うもの） |
|---|---|---|
| **フロントエンド（frontend）** | アプリの画面側、ユーザーが直接操作する部分 | TypeScript / React / React Native / Expo |
| **バックエンド（backend）** | サーバー側、データの保存や認証などを担当 | Supabase（自動でAPIを生成） |
| **データベース（database）** | データを保存しておく場所 | PostgreSQL（Supabase内蔵） |

> **本書のポイント:** 通常はバックエンドを自分でゼロから作る必要がありますが、**Supabase（スーパベース）** がそれを肩代わりしてくれます。だから「アプリの画面づくりに集中していれば、結果としてデータ保存もできるアプリが完成する」というお得な構成になっています。

## 0.6 コードを書く・動かすための道具

このチュートリアルで実際に使う道具（ツール）の名前を、先に頭出ししておきます。詳しいインストール方法は第1章で説明します。

| 道具 | 役割 | 例えるなら |
|---|---|---|
| **VS Code** | コードを書くためのエディタ（高機能なメモ帳） | 原稿用紙＋校閲機能 |
| **ターミナル**（PowerShell / Terminal） | キーボードから命令文を打ってPCを操作する画面 | レストランでの口頭注文 |
| **Node.js**（ノードジェイエス） | JavaScript/TypeScript をPC上で動かす実行環境 | 家庭用キッチン |
| **npm**（エヌピーエム） | 他人が作った便利な部品（パッケージ）を入れて管理するツール | アマゾン的な部品通販 |
| **Expo**（エクスポ） | React Native アプリ作りを劇的に簡単にする道具一式 | アプリ開発のスターターキット |
| **Expo Go**（エクスポ・ゴー） | 作りかけアプリを自分のスマホで即確認できる無料アプリ | 試着室 |
| **Git**（ギット） | 「いつ・誰が・何を変えたか」を記録するバージョン管理ツール | 編集履歴付きノート |
| **GitHub**（ギットハブ） | Gitで管理したコードをインターネット上に保管する場所 | コード版のクラウドストレージ |

> **ターミナルの「プロンプト記号」について:** ターミナルを開くと、命令を待ち受ける目印（プロンプト、prompt）として `$` 、`>` 、`%` などが行頭に出ます。これは「ここから先に命令を打ってね」というマークで、命令の一部ではありません。本書のサンプルで `$ npm install` と書かれていたら、`$` は打ち込まずに `npm install` の部分だけを入力します。

## 0.7 「実機・シミュレータ・エミュレータ」って何？

スマホアプリは、作っている途中で**何度も動作確認**します。確認する方法は3つあります。

| 方法 | 何か | 必要なもの |
| --- | --- | --- |
| **実機（じっき）** | あなたの本物のiPhone/Android端末で動かす | スマホ＋Expo Goアプリ（一番手軽） |
| **iOSシミュレータ** | Mac の中に iPhone の画面を再現して動かす | **Macが必須** |
| **Androidエミュレータ** | PC の中に Android スマホを再現して動かす | Android Studio（WindowsでもMacでも可） |

> **「シミュレータ」と「エミュレータ」の違い（豆知識）**：細かい技術的違いはありますが、初心者の理解としては**「どちらもPCの画面の中に映し出される"偽物のスマホ"」**でOKです。Apple は「シミュレータ」、Google は「エミュレータ」と呼ぶ、という名前の違い程度に考えてください。本書では一番手軽な**実機＋Expo Go**を中心に進めます。

### 0.8 「コマンドを実行する」ってどういうこと？

このチュートリアルでは何度も**「このコマンドを実行してみましょう」**と書きます。「コマンド（command）」とは「命令」、「実行」とは「その命令をコンピュータに動かしてもらうこと」です。ターミナルを開いて、たとえば次のように入力して Enter キーを押します。

コピー

```
npx expo start
# ↑ この一行が「コマンド」。意味を分解すると…
# npx        : npm に付属する「パッケージを一時的にダウンロードして即実行する」コマンド
# （半角スペース）: コマンドと引数の区切り。スペースは必ず半角（全角だとエラーになる）
# expo       : 実行したい道具（Expoのコマンドラインツール）の名前
# start      : expo に対する指示。「開発用サーバーを起動して」という意味のサブコマンド
# Enter を押すと、Expoが起動して「アプリを確認するためのQRコード」が表示される
```

> **「npx」と「npm」の違い:** どちらも Node.js に付いてくるコマンドです。`npm install` は「部品をPCに保存（インストール）する」、`npx` は「その場限りで取ってきて実行し、終わったら基本残さない」イメージです。第1章で実際に使いながら覚えます。

ここまでが「読む前の超基礎」です。すべてを覚える必要はなく、**「分からない単語が出てきたらこの 0 章に戻ってくる」**という辞書のように使ってください。

---

## 1. この教材の目的と対象読者

### 目的

この教材は、以下のことを目的としています。

- **スマホアプリ開発の基礎**を、手を動かしながら体系的に学ぶ

- **TypeScript + React Native + Expo + Supabase** という、実務でも広く使われている技術スタック（technology stack：技術の組み合わせ）を習得する
- 一つのアプリを最初から最後まで作り切り、最終的に**アプリストアに公開する**ことで、開発の全体像を理解する
- CRUD（クラッド。Create=作成、Read=読み取り、Update=更新、Delete=削除の頭文字で、データ操作の基本4つ）操作を通じて、**アプリ開発の基本パターン**を身につける

> **「CRUD」とは？** ほぼ全てのアプリは、データの「作成・読み取り・更新・削除」という4つの操作で成り立っています。SNSなら投稿の作成・表示・編集・削除、家計簿アプリなら支出の登録・一覧・修正・削除です。この4つをマスターすれば、どんなアプリでも基本は作れます。
>
> **Web版チュートリアルとの関係:** この教材には姉妹編として `next` （Next.jsでWebアプリを作る）フォルダがあります。同じ「書籍管理アプリ」を題材にしているので、**同じSupabaseバックエンドをWeb版とモバイル版の両方から使う**ことができます。「同じデータを、ブラウザでもスマホアプリでも扱える」という現代的な開発を体験できます。

### 対象読者

| 対象 | 説明 |
|---|---|
| **完全な初心者** | プログラミング自体がほぼ初めての方。本書はこの層を最優先に書いています |
| **Web開発経験者** | HTML/CSS/JavaScript は分かるが、スマホアプリは未経験の方 |
| **他言語からの転向者** | Python や Java などの経験があり、アプリ開発を始めたい方 |
| **個人開発者・学生** | 自分のアイデアをストアに出してみたい方 |

> **安心してください！** 分からないことがあっても、各章で丁寧に説明します。**初めて出てくるコードやコマンドには、その都度すべて解説**を入れています。エラーが出ても慌てず、一歩ずつ進めていきましょう。

---

## 2. React Native とは何か

書籍管理アプリを作り始める前に、これから使う **React Native** がどういうものかを理解しておきましょう。ここで全てを覚える必要はありません。「ふーん、こういう仕組みなんだ」くらいで十分です。

### 2.1 「1つのコードで iPhone と Android 両方」が作れる仕組み

通常、iPhone アプリと Android アプリは**別々の言語**で作ります。

- iPhone（iOS）アプリ → **Swift（スウィフト）** という言語
- Android アプリ → **Kotlin（コトリン）** という言語

つまり本来は、同じアプリを2回（2言語で）作る必要があります。これは大変です。

**React Native（リアクト・ネイティブ）** は、**TypeScript（1つの言語）で書いたコードを、iPhone でも Android でも動くアプリに変換してくれる**技術です。Meta社（旧Facebook、InstagramやWhatsAppを運営）が開発しました。

コピー

```
                          ┌→ iPhone（iOS）アプリ
TypeScriptで1回書く ──→ React Native ─┤
                          └→ Android アプリ
```

> **「クロスプラットフォーム」とは？**「プラットフォーム（platform）」は「土台＝iOS や Android のこと」、「クロス（cross）」は「横断する」という意味。**1つのコードで複数のOS向けアプリを作れること**を「クロスプラットフォーム開発」と呼びます。React Native はその代表格です。

### 2.2 React（リアクト）との関係

名前に「React」と入っている通り、React Native は **React（リアクト）** という技術がベースです。

- **React** … もともとは**Webサイトの画面**を部品（コンポーネント）の組み合わせで作るための技術
- **React Native** … その React の考え方を**スマホアプリの画面**に応用したもの

つまり、React の「コンポーネント（画面部品）を組み合わせて画面を作る」という考え方を学べば、それがそのままスマホアプリ作りに活かせます。本書では第3章で React、第4章で React Native を学びます。

> **WebのHTMLとの違い:** Webでは `<div>` や `<p>` というHTMLタグで画面を作りますが、スマホアプリにはブラウザがないのでHTMLタグは使えません。代わりに React Native では `<View>`（箱）や `<Text>`（文字）といった**専用の部品**を使います。「名前が違うだけで考え方は同じ」と理解してください（第4章で対比表を載せます）。

---

## 3. 完成イメージ

このチュートリアルで作成する「書籍管理アプリ」は、以下のような画面構成のスマホアプリになります。

## 書籍一覧画面


書籍管理
タイトルや著者で検索...
リーダブルコード
著者: Dustin Boswell
読了
プロを目指す人のためのTypeScript入門
著者: 鈴木 僚太
読書中
達人プログラマー
著者: David Thomas
未読

### 書籍登録・編集画面




### 主な機能

- **一覧表示**: 登録した書籍をリスト形式で表示（スクロール対応）
- **新規登録**: タイトル・著者・ステータス・メモを入力して書籍を登録
- **編集**: 登録済みの書籍情報を変更
- **削除**: 不要な書籍データを削除（確認ダイアログ付き）
- **検索**: タイトルや著者名で書籍を絞り込み
- **ストア公開**: 完成したアプリを App Store / Google Play に申請

---

## 4. 学習ロードマップ

以下のステップで、段階的にアプリを完成させ、最終的に公開します。焦らず、一つずつ進めましょう。「ロードマップ（roadmap）」は「道筋を示した地図」という意味の英単語です。

**第0章: はじめに（この章）**
全体概要・技術スタック・学習ロードマップ

↓

**第1章: 開発環境の構築**
Node.js / VS Code / Expo（Expo無し構成も解説）

↓

**第2章: TypeScript入門**
型の基本を学ぶ

↓

**第3章: React入門**
コンポーネントの考え方

↓

**第4章: React Native / Expo入門**
スマホ画面の部品とルーティング

↓

**第5章: Supabaseのセットアップ**
データベースを準備する（DB選択肢の比較）

↓

**第6章: プロジェクトのセットアップ**
画面遷移（ナビゲーション）を作る

↓

**第7・8章: CRUD操作の実装**
一覧・作成・編集・削除

↓

**第9章: スタイリングとUI**
NativeWindで見た目を整える

↓

🚀

**第10章: アプリの公開**

ビルドしてストアに申請する

↓

🎉

**完成・公開！**

あなたのアプリがストアに並びます。おめでとうございます！

各章の目安学習時間は **30分〜1時間程度** です。全体を通して、**約2〜3週間** で完走できる構成になっています（ストア審査の待ち時間を除く）。

---

## 5. 使用技術スタック一覧

このチュートリアルで使用する技術の一覧と、それぞれの役割をまとめます。「スタック（stack）」は「積み重ね」という意味で、複数の技術を組み合わせて使うことを指します。

| 技術 | バージョン目安 | カテゴリ | 役割 |
| --- | --- | --- | --- |
| **TypeScript** | 5.x | プログラミング言語 | JavaScriptに「型」（データの種類の指定）を追加した言語。コードの安全性と可読性を向上させる |
| **React** | 19.x | UIライブラリ | 画面を「コンポーネント（部品）」の組み合わせで作るための土台。Meta社が開発 |
| **React Native** | 0.7x | モバイルフレームワーク | TypeScriptで書いたコードをiOS/Androidアプリに変換する技術。Meta社が開発 |
| **Expo** | SDK 52+ | 開発プラットフォーム | React Native開発を簡単にする道具一式。ビルドや公開も支援する |
| **Expo Router** | 4.x | 画面遷移（ルーティング） | ファイルを置くだけで画面遷移が作れる仕組み。Next.jsと同じ考え方 |
| **Supabase** | - | BaaS（バックエンドのサービス） | データベース・認証・APIを提供。バックエンドを自前で作る手間を省ける |
| **NativeWind** | 4.x | スタイリング | Tailwind CSSの記法でReact Nativeの見た目を整えるライブラリ |
| **EAS** | - | ビルド／公開サービス | Expoが提供する、ストア提出用アプリを作るクラウドサービス |
| **Node.js** | 20.x 以上 | ランタイム（実行環境） | JavaScript/TypeScriptをPC上で実行する環境。開発ツールの動作に必要 |
| **npm** | 10.x | パッケージマネージャ | 他の開発者が作った部品をインストール・管理するツール |
| **Git / GitHub** | 2.x | バージョン管理 | コードの変更履歴を記録・共有するツールとサービス |
| **VS Code** | 最新版 | コードエディタ | コードを書くためのエディタ。Microsoft社が開発 |

> **「バージョン」って何？** ソフトウェアは改良されるたびに番号が振り直されます。「5.x」と書いてあれば「メジャー番号が 5 の系列ならどの細かい版でも OK」という意味です。`x` は「何でもいい」を表すワイルドカードです。表記は SemVer（セマンティックバージョニング）と呼ばれ、`メジャー.マイナー.パッチ` の3階層が標準です。Expo は独自に「SDK 52」のような"SDKバージョン"も使います。

## 各技術のひとこと概要

### TypeScript — 「型」のある安全な言語

「この変数には文字列しか入らない」「この関数は数値を返す」といったルール（＝型）をコードに書ける言語です。間違いをアプリ実行前に発見できます。詳しくは第2章。

### React / React Native — 部品で画面を組み立てる

画面を**コンポーネント（部品）** に分割して作ります。「書籍カード」という部品を一度作れば、一覧画面で何回でも使い回せます。レゴブロックのイメージです。詳しくは第3・4章。

### Expo — React Native開発を簡単にする"スターターキット"

React Native だけでアプリを作ろうとすると、iOS/Android の複雑な設定（Xcode、Android Studio の難しい設定）が必要です。**Expo** はそれらを肩代わりし、「コマンド一発でプロジェクト作成」「QRコードを読むだけで実機確認」「クラウドでストア提出用ビルド」を可能にします。詳しくは第1章。

### Supabase — バックエンドを手軽に用意

データベースやユーザー認証などの**バックエンド機能**をクラウドで提供するサービスです。自分でサーバーを構築せずに、データの保存・取得ができます。詳しくは第5章。

### NativeWind — Tailwindの書き方でスマホUIを装飾

`className="text-lg font-bold text-blue-600"` のような短い指定で見た目を整えられます。Web版チュートリアルのTailwind CSSと同じ書き方なので学習が地続きです。詳しくは第9章。

---

## 6. なぜこの技術スタックを選んだのか（選択肢の比較）

技術にはたくさんの選択肢があります。「**どんな場合にどの技術を選ぶべきか**」を、他の選択肢と比較しながら解説します。これは本書全体で大切にしている視点です。

### 6.1 アプリ開発手法の比較: React Native vs Flutter vs ネイティブ vs PWA

スマホアプリを作る方法は React Native だけではありません。代表的な4つを比べます。

| 観点 | React Native | Flutter | ネイティブ（Swift/Kotlin） | PWA |
|---|---|---|---|---|
| 言語 | TypeScript/JavaScript | Dart | Swift（iOS）/ Kotlin（Android） | HTML/CSS/JS |
| iOS/Android両対応 | ◯ 1コードで両方 | ◯ 1コードで両方 | × OSごとに別々 | ◯（ブラウザ依存） |
| 学習コスト | 低〜中（Web知識が活きる） | 中（Dartを新規に学ぶ） | 高（2言語＋2環境） | 低 |
| 動作の速さ・滑らかさ | 高い | 非常に高い | 最高 | 中 |
| スマホ機能の利用 | 豊富（カメラ・通知など） | 豊富 | 全機能 | 限定的 |
| 求人・実績 | 非常に多い | 増加中 | 多い | 多い |
| ストア公開 | ◯ | ◯ | ◯ | △（ストアに出しにくい） |

> **用語整理:** - **Flutter（フラッター）**：Googleが開発したクロスプラットフォーム技術。Dart（ダート）という言語を使う。 - **PWA（Progressive Web App）**：Webサイトをアプリのように使える技術。インストール風に使えるが、ストア公開や高度なスマホ機能の利用には制約がある。

**React Nativeを選んだ理由:** Web開発で広く使われる **React/TypeScript の知識がそのまま活かせる**こと、求人・実務実績が非常に多いこと、そして本書のWeb版チュートリアル（Next.js）と**ほぼ同じ考え方・同じバックエンド**で学べることが決め手です。「Webもモバイルも両方できる人材」を最短で目指せます。

> **どんな時にどれを選ぶ？** - **React Native** → Web経験がある／チームがJS/TSに慣れている／Webと共通化したい - **Flutter** → 動作の滑らかさを最優先／デザインを細部までこだわりたい - **ネイティブ** → ゲームや高度なカメラ処理など最高性能が必要／片方のOSだけで良い - **PWA** → とにかく手軽に／ストア公開は必須でない

## 6.2 React Nativeの開発環境: Expo vs React Native CLI

React Native でアプリを作るとき、土台の作り方は大きく2通りあります。**本書はExpoを採用**しますが、Expoを使わない方法（React Native CLI）も第1章で参考解説し、メリット・デメリット・切り替え方も説明します。

| 観点 | Expo（本書採用） | React Native CLI（素のRN） |
|---|---|---|
| **環境構築の手間** | 非常に少ない（数分で開始） | 多い（Xcode/Android Studioの設定必須） |
| **実機確認** | Expo Goアプリで即確認（QR読むだけ） | エミュレータ/実機ビルドが必要 |
| **ネイティブ機能** | Expoが用意した範囲＋追加も可能 | 何でも自由に組み込める |
| **ビルド（ストア提出用）** | EAS（クラウド）で簡単 | 自分でXcode/Android Studioでビルド |
| **Macの要否** | iOSビルドもクラウドで可能（Mac不要） | iOSビルドにMacが必須 |
| **自由度** | やや制約あり（だが十分広い） | 最大（OSの全機能に手が届く） |
| **初心者向き** | ◎ とても向いている | △ ハードルが高い |

> 「**CLI**」とは？ Command Line Interface（コマンドライン・インターフェース）の略。「ターミナルにコマンドを打って操作する道具」のこと。`React Native CLI` は「素のReact Nativeをコマンドで操作する公式ツール」を指します。

**Expoを選んだ理由:** 完全初心者にとって、Xcode や Android Studio の複雑な初期設定なしに、**数分でアプリ作りを始められ、自分のスマホですぐ動作確認でき、Mac が無くても iOS アプリをビルドできる**ことは、挫折を防ぐ最大の助けになります。

> 「**昔のExpoは機能制限が…」という不安について:** 以前のExpoは「使えるネイティブ機能が限られる」という弱点がありましたが、現在は **Config Plugin（コンフィグ・プラグイン）** や **Development Build（開発ビルド）** という仕組みで、ほぼ何でも組み込めるようになっています。さらに、後から「`npx expo prebuild` というコマンド1つでCLI構成（bare workflow）に移行する」道も残されています。つまり「**まずExpoで始めて、本当に必要になったらCLIへ切り替える**」のが現代の定石です（詳しい手順は第1章）。

## 6.3 言語の比較: TypeScript vs JavaScript

| 観点 | TypeScript | JavaScript |
|---|---|---|
| **型安全性** | あり（実行前にエラー検出） | なし（実行時にエラー発覚） |
| **コードの可読性** | 高い（型が仕様書の役割） | 普通 |
| **学習コスト** | やや高め（型の学習が必要） | 低め |
| **開発効率** | 高い（エディタの補完が強力） | 普通 |
| **エラーの発見しやすさ** | 書いている途中で気づける | 実行してから気づく |

**TypeScriptを選んだ理由:** 最初は少し学ぶことが増えますが、型があることでエディタが強力に補助してくれるため、結果的に**初心者にもやさしい**言語です。「何を渡せばいいか分からない」という場面が大幅に減ります。

## 6.4 バックエンド／DBの比較: Supabase vs Firebase vs ローカルDB

スマホアプリのデータ保存先には複数の選択肢があります。本書は**Supabase**を採用しますが、選択の指針を示します（詳細は第5章）。

| 観点 | Supabase | Firebase | ローカルDB（SQLite等） |
|---|---|---|---|
| **データの種類** | PostgreSQL（リレーショナルDB） | Firestore（NoSQL） | 端末内SQLite |
| **データの置き場所** | クラウド（サーバー） | クラウド（サーバー） | スマホ端末内のみ |
| **複数端末で同じデータ** | ◯ できる | ◯ できる | × できない |
| **オフライン動作** | △ 工夫が必要 | ◯ 得意 | ◎ 完全オフラインOK |
| **SQLの学習** | できる | できない | できる |
| **無料枠** | 十分（個人開発に最適） | 十分 | 不要（端末内なので） |
| **Web版との共通利用** | ◎ 本書のNext.js版と共通 | ◯ | × |

> **「リレーショナルDB / NoSQL」とは？** - リレーショナルDB（RDB）：表（テーブル）と表の関係でデータを管理する方式。SQLで操作する。PostgreSQL、MySQL など。 - NoSQL：表形式に縛られないデータベースの総称。Firestore は「ドキュメント型」のNoSQL。

**Supabaseを選んだ理由:** PostgreSQL（業界標準のDB）を使うので、学んだSQL知識が他でも活きること、Web版チュートリアルと**同じバックエンドを共用できる**こと、無料枠が個人学習に十分なことが理由です。

> **どんな時にどれを選ぶ？** - **Supabase** → 複数端末で同期したい／SQLを学びたい／Webと共通化したい - **Firebase** → リアルタイム同期やオフラインを重視／Googleサービスと連携したい - **ローカルDB** → ネット不要の完全オフラインアプリ（例: 自分専用のメモ・電卓）を作りたい

## 6.5 スタイリングの比較: NativeWind vs StyleSheet vs UIキット

アプリの「見た目（色・大きさ・配置）」を指定する方法も複数あります。本書は**NativeWind**を採用します（詳細は第9章）。

| 観点 | NativeWind | StyleSheet（RN標準） | UIキット（Paper等） |
|---|---|---|---|
| **記述量** | 少ない（短いクラス名） | 多い（オブジェクトで指定） | 少ない（完成部品を使う） |
| **学習コスト** | 低（Tailwindと同じ） | 低（追加ライブラリ不要） | 中（各キットの作法を学ぶ） |
| **デザインの自由度** | 高い | 高い | 中（キットの範囲内） |
| **Web版との共通性** | ◎ TailwindでWebと共通 | △ | △ |

**NativeWindを選んだ理由:** Web版チュートリアルで学ぶ Tailwind CSS とほぼ同じ書き方で、記述量が少なく、デザインの自由度も高いためです。

> **どんな時にどれを選ぶ？** - **NativeWind** → Tailwind経験がある／素早くきれいに書きたい - **StyleSheet** → 追加ライブラリを増やしたくない／RNの基本を厳密に学びたい - **UIキット** → デザインを自分で考えず、完成された部品で素早く作りたい

---

## 7. 前提知識

このチュートリアルを始めるにあたって、以下の知識があると学習がスムーズですが、**無くても本書の説明だけで進められる**ように書いています。

### あると望ましい知識（なくても大丈夫です）

- **プログラミングの基本概念**: 変数、条件分岐（if文）、繰り返し（for文）、関数がどういうものか（→ 第2章で基礎から説明します）
- **HTML/CSS の基本**: タグや「色・配置の指定」の感覚（→ React Nativeは似た考え方なので理解が早まります）
- **コマンドライン操作**: ターミナルで `cd` （フォルダ移動）が使えること（→ 第1章で説明します）

### 必要なもの（ハードウェア・アカウント）

- **PC**: Windows でも Mac でも可（**iOSアプリの確認・公開も、Expoのおかげで基本Macは不要**）
- **スマートフォン**: iPhone か Android（実機確認に使用。無くてもエミュレータで代替可）
- **メールアドレス**: Expo / Supabase / GitHub などの無料アカウント登録に使用
- （**公開時のみ**）**開発者アカウント費用**: Apple Developer（年 約$99）、Google Play（**初回のみ** $25）。第10章で詳説します

> **前提知識に不安がある方へ:** 心配しなくても大丈夫です。本書は**完全な初心者**を想定し、初めて出てくるコードやコマンドにはすべて解説を入れています。分からない部分があったら、飛ばさずにゆっくり読み返してみてください。

---

## 8. この教材で学べること

チュートリアルを完走すると、以下のスキルが身につきます。一つずつチェックを付けながら進めましょう！（Markdownの `- [ ]` は「未完了のチェックボックス」を表す書き方です）

## 開発環境

- [ ] Node.js と npm のインストールと基本操作
- [ ] VS Code のセットアップと便利な拡張機能の導入
- [ ] Expo を使ったプロジェクトの作成と実機確認
- [ ] （参考）Expoを使わない React Native CLI 環境の構築と切り替え方

## TypeScript / React

- [ ] 基本的な型（`string`, `number`, `boolean`）の使い方
- [ ] コンポーネント・props・state・Hooks の基本

## React Native / Expo

- [ ] `View` / `Text` / `Image` / `FlatList` などのコア部品の使い方
- [ ] Flexbox によるレイアウト
- [ ] Expo Router によるファイルベースの画面遷移

## バックエンドとデータ操作

- [ ] Supabase のセットアップとアプリからの接続
- [ ] CRUD（作成・読取・更新・削除）の実装

## スタイリングと公開

- [ ] NativeWind による見た目の調整
- [ ] EAS によるビルドと、App Store / Google Play への公開

> **準備はいいですか？** それでは、第1章で開発環境を整えるところから始めましょう！分からないことがあれば、いつでもこの第0章に戻ってきてください。